

Section A: Scope & Lexical Scope (1–10)

☐ Q1 [Status: Pending]

Predict Output

```
let a = 10;

function outer() {
  let a = 20;

  function inner() {
    console.log(a);
  }

  inner();
}

outer();
```

Section A (Cont.): Scope & Lexical Scope

☐ Q2 [Status: Pending]

```
let count = 1;

function test() {
  console.log(count);

  let count = 2;
}

test();
```

☐ Q3 [Status: Pending]

Create a nested function structure of 4 levels and print variables from all parent scopes.

Expected:

```
1
2
3
```

☐ **Q4 [Status: Pending]**

Write a function that returns another function and demonstrates lexical scope.

☐ **Q5 [Status: Pending]**

Predict Output

```
var x = 100;
```

```
function a() {
```

```
    console.log(x);
```

```
    var x = 50;
```

```
}
```

```
a();
```

☐ **Q6 [Status: Pending]**

Create a function where a child function can access grandparent variables.

☐ **Q7 [Status: Pending]**

Predict Output

```
let name = "Ram";
```

```
function show() {
```

```
    let name = "Mohan";
```

```
    return function() {
```

```
        console.log(name);
```

```
    }
```

```
}
```

```
show()();
```

☐ **Q8 [Status: Pending]**

Create a 3-level nested function and access all variables inside the innermost function.

☐ **Q9 [Status: Pending]**

Can a parent access a child variable?
Demonstrate using code.

☐ **Q10 [Status: Pending]**

Create a private variable using lexical scope.

Section B: Closures (11–20)

☐ **Q11 [Status: Pending]**

Create a counter closure.

Output:

1
2
3

Q12

Create a reverse counter.

Output:

10
9
8

Q13

Create a closure-based bank account.

Methods:

deposit()
withdraw()
checkBalance()

Q14

Create a closure that remembers the username forever.

Q15

Create a closure-based login attempt tracker.

After 3 failed attempts:

Account Locked

Q16

Build a closure that stores a secret password.

Only getter should be available.

Q17

Build a shopping cart using closure.

Methods:

addItem()
removeItem()
showItems()

Q18

Create a closure-based voting machine.

Each user can vote only once.

Q19

Build a closure that counts how many times a function is executed.

Q20

Build a closure-based expense tracker.

Methods:

addExpense()
getTotal()

Section C: IIFE (21–25)**Q21**

Create an IIFE that prints:

Welcome to JavaScript

Q22

Pass arguments to an IIFE and calculate area of rectangle.

Q23

Create an arrow function IIFE.

Q24

Use IIFE to create private variables.

Q25

Create a module pattern using IIFE.

Methods:

increment()
decrement()
reset()

Section D: Higher Order Functions (26–30)

Q26

Create a function:

calculate(a,b,operation)

Perform:

add
subtract
multiply
divide

Q27

Create your own custom HOF named:

repeat()

Example

repeat(5, sayHello)

Q28

Create a logger HOF.

Output:

[INFO] User Logged In

Q29

Create a function execution timer using HOF.

Output:

Execution Time: XX ms

Q30

Create a function wrapper that counts how many times another function was called.

Section E: map() (31–33)

Q31

Given

```
const users = [  
  {name:"Ram",age:25},  
  {name:"Mohan",age:30},  
  {name:"Amit",age:22}  
];
```

Return:

```
[  
  "Ram",  
  "Mohan",  
  "Amit"  
]
```

using map.

Q32

Convert

[1,2,3,4,5]

into

[1,4,9,16,25]

using map.

Q33

Create an array of product names from:

```
[  
  {name:"Laptop"},  
  {name:"Mouse"},  
  {name:"Keyboard"}  
]
```

Section F: filter() (34–36)

Q34

Filter all even numbers.

Input:

[1,2,3,4,5,6]

Output:

[2,4,6]

Q35

Filter users whose age is greater than 25.

Q36

Filter expensive products.

Condition:

price > 1000

Section G: reduce() (37–40)

Q37

Find total sum.

Input:

[10,20,30,40]

Output:

100

Q38

Find maximum value using reduce.

Input:

[5,12,3,45,2]

Output:

45

Q39

Count total characters.

Input:

["JavaScript","React","Node"]

Output:

20

Q40 ★ Ultra Hard

Given:

```
const orders = [  
  {  
    id:1,  
    amount:5000,  
    status:"completed"  
  },  
  {  
    id:2,  
    amount:2000,  
    status:"pending"  
  },  
  {  
    id:3,  
    amount:7000,  
    status:"completed"  
  },  
  {  
    id:4,  
    amount:1000,  
    status:"completed"  
  }  
];
```

Using only:

filter()
map()
reduce()

Find:

1. Completed Orders
 2. Total Revenue
 3. Average Revenue
 4. Highest Order Amount
 5. Array of Order IDs
-